

Motivation Efficient VLA inference without retraining



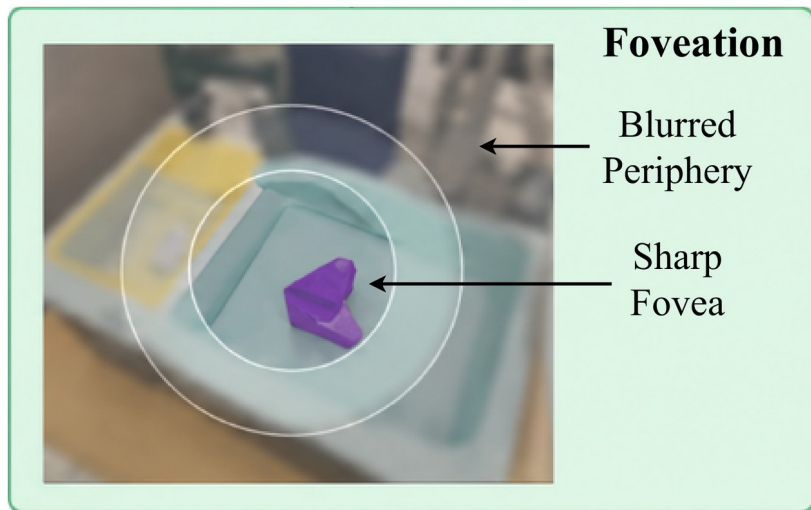
The control loop of a frozen VLA policy (Fig. 2 of the paper). Green, orange, purple, and blue boxes mark where the five tricks enter; Redundancy Control is action repeat.

[1] VLA inference is slow. Tens to hundreds of milliseconds per step (60 to 450 ms in our runs), which limits how often the policy can observe, decide, and correct.

[2] Gains are reported in isolation. Training-free tricks are typically evaluated for a specific trick, backbone, benchmark, and implementation, so transfer across models is unknown.

Do the gains transfer across VLA models and environments, **or are they configuration-dependent?**

Trick 1 Visual foveation



- Sharp central disc, progressively blurred periphery
- Keep ratio 20% or 50%; token count unchanged, no latency saving
- Tests whether high-frequency detail is needed only near the objects and the gripper

What to see

Original

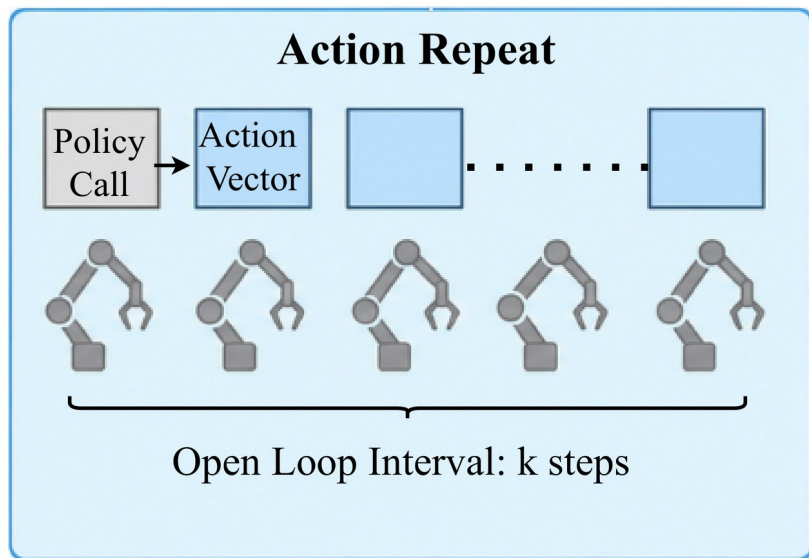


Foveation 20%



CogACT on WidowX, Eggplant in basket, same episode, 2× speed.
Green border = success, red = failure.

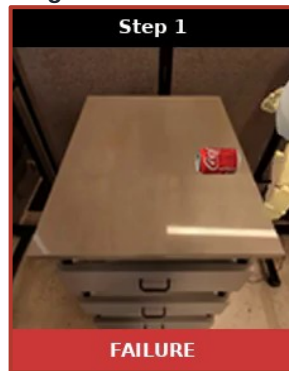
Trick 2 Action repeat



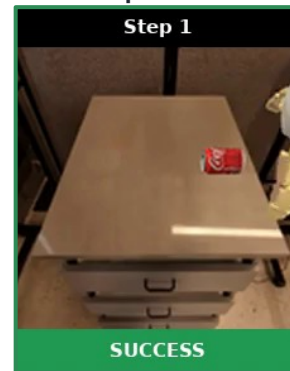
- Hold each predicted action for $k = 2$ or 4 steps, about one call per k steps
- Cuts policy calls by about k , but lengthens the open-loop interval
- Fixed reduction, no state check: a speed-oriented baseline

When to act

Original

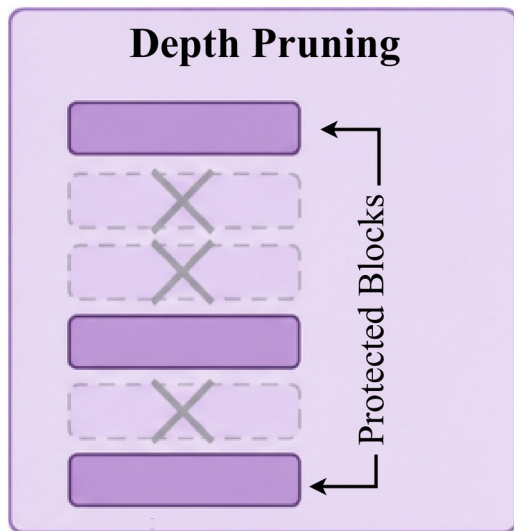


Action repeat 4



OpenVLA on Fractal, Pick coke can, same episode, 1.5× speed.
Green border = success, red = failure.

Trick 3 Depth pruning



- Score each decoder block by Block Influence
- Remove 1, 2, or 4 low-influence blocks; none adjacent, early blocks and the final block kept
- Fixed once from calibration data, not adapted at test time

How much to compute

Original



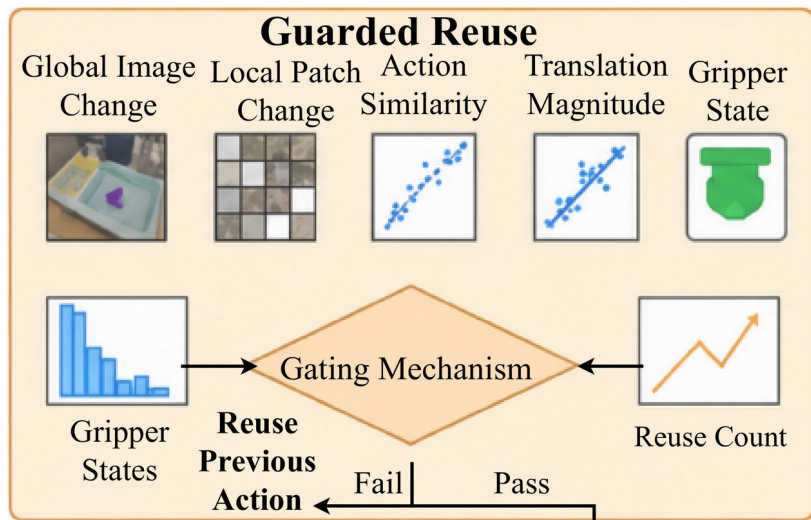
Depth pruning 1



CogACT on WidowX, Stack cube, same episode, 2× speed. Green border = success, red = failure.

Trick 4 Guarded action reuse

When to act



- Reuse the previous action only when all gates pass: small global and local image change, recent actions agree, translation not near zero, gripper unchanged, reuse count below a cap
- Presets: strict, moderate, aggressive

Original



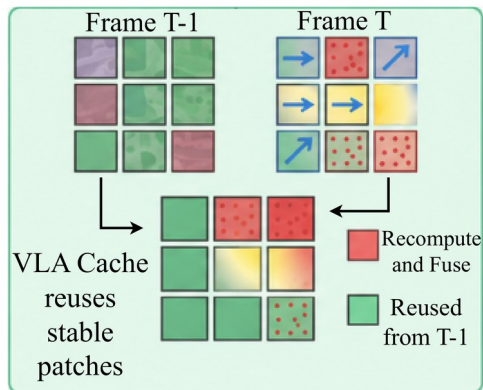
Reuse moderate



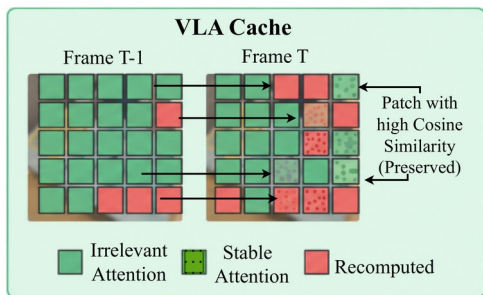
CogACT on WidowX, Carrot on plate, same episode, 2× speed.
Green border = success, red = failure.

Trick 5 Temporal fusion

How much to compute

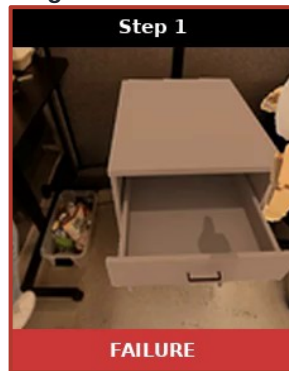


- Reuse visual tokens of stable patches from the previous call
- Recompute patches with motion, local structure, language attention, or in the interaction region
- Presets: motion-entropy, task-aware, conservative-adaptive

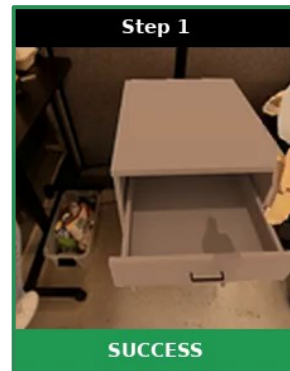


Top: fusion mask. Bottom: VLA-Cache criterion.

Original



Fusion task-aware



OpenVLA on Fractal, Close drawer, same episode, 1.5× speed. Green border = success, red = failure.

CogACT, WidowX: Eggplant in basket

All 14 configurations on the same episode (same initial state), 3× speed. Green border = success, red = failure.

Original



Foveation 20%



Foveation 50%



Action repeat 2



Action repeat 4



Depth pruning 1



Depth pruning 2



Depth pruning 4



Reuse strict



Reuse moderate



Reuse aggressive



Fusion motion



Fusion task-aware



Fusion conserv.



CogACT, WidowX: Stack cube

All 14 configurations on the same episode (same initial state), 2× speed. Green border = success, red = failure.

Original



Foveation 20%



Foveation 50%



Action repeat 2



Action repeat 4



Depth pruning 1



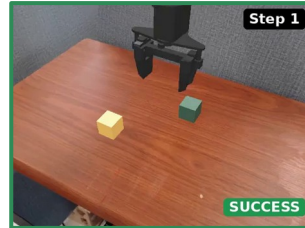
Depth pruning 2



Depth pruning 4



Reuse strict



Reuse moderate



Reuse aggressive



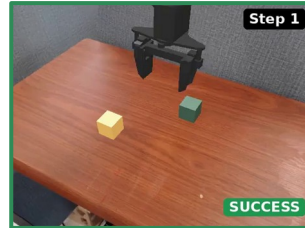
Fusion motion



Fusion task-aware



Fusion conserv.



CogACT, WidowX: Spoon on towel

All 14 configurations on the same episode (same initial state), 2× speed. Green border = success, red = failure.

Original



Foveation 20%



Foveation 50%



Action repeat 2



Action repeat 4



Depth pruning 1



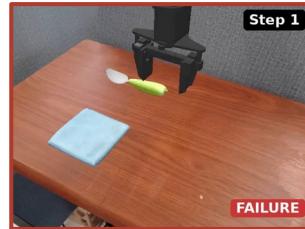
Depth pruning 2



Depth pruning 4



Reuse strict



Reuse moderate



Reuse aggressive



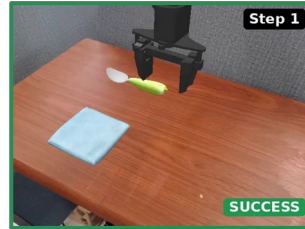
Fusion motion



Fusion task-aware

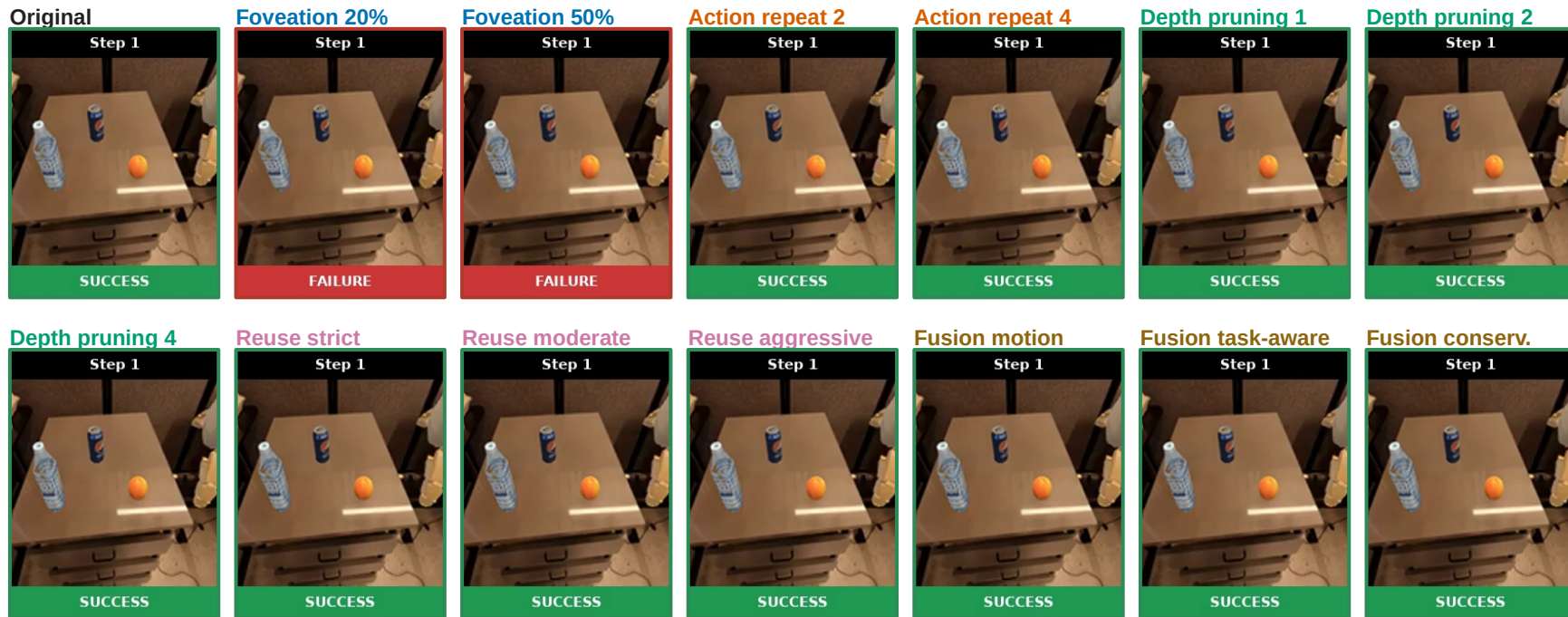


Fusion conserv.



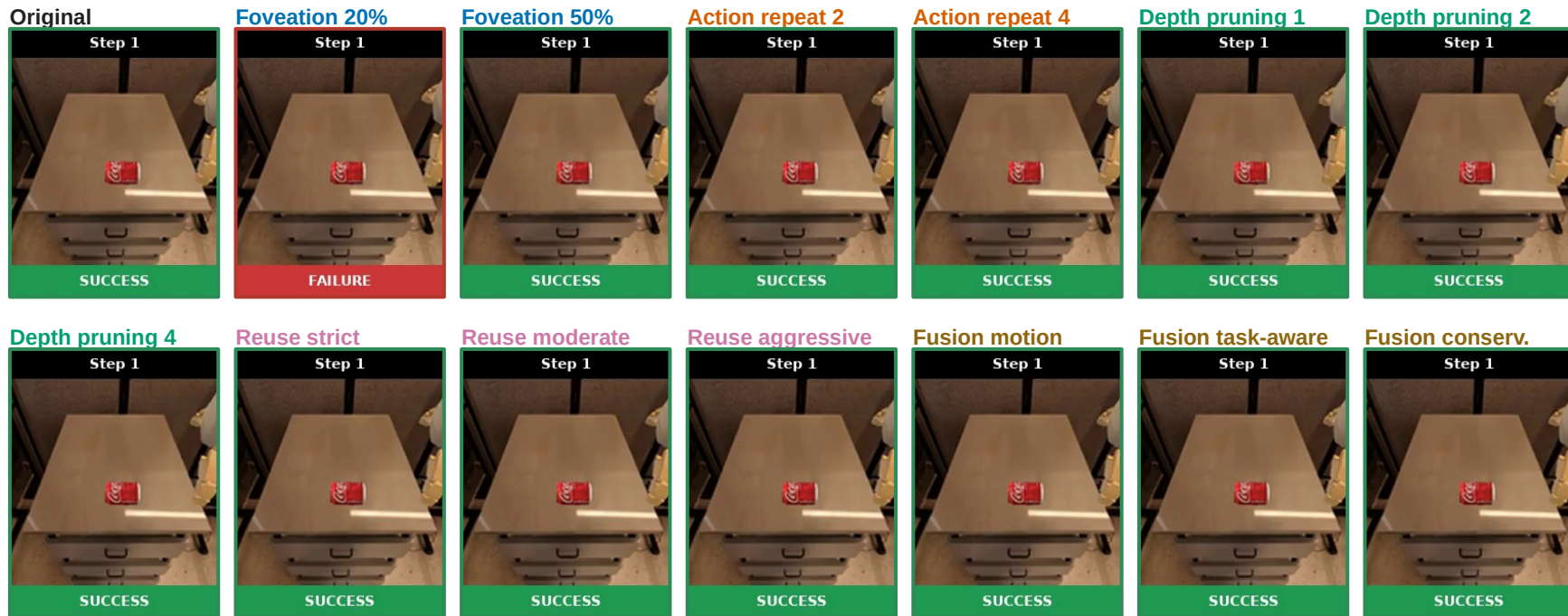
OpenVLA, Fractal: Move near

All 14 configurations on the same episode (same initial state), 2× speed. Green border = success, red = failure.



OpenVLA, Fractal: Pick coke can

All 14 configurations on the same episode (same initial state), 2× speed. Green border = success, red = failure.



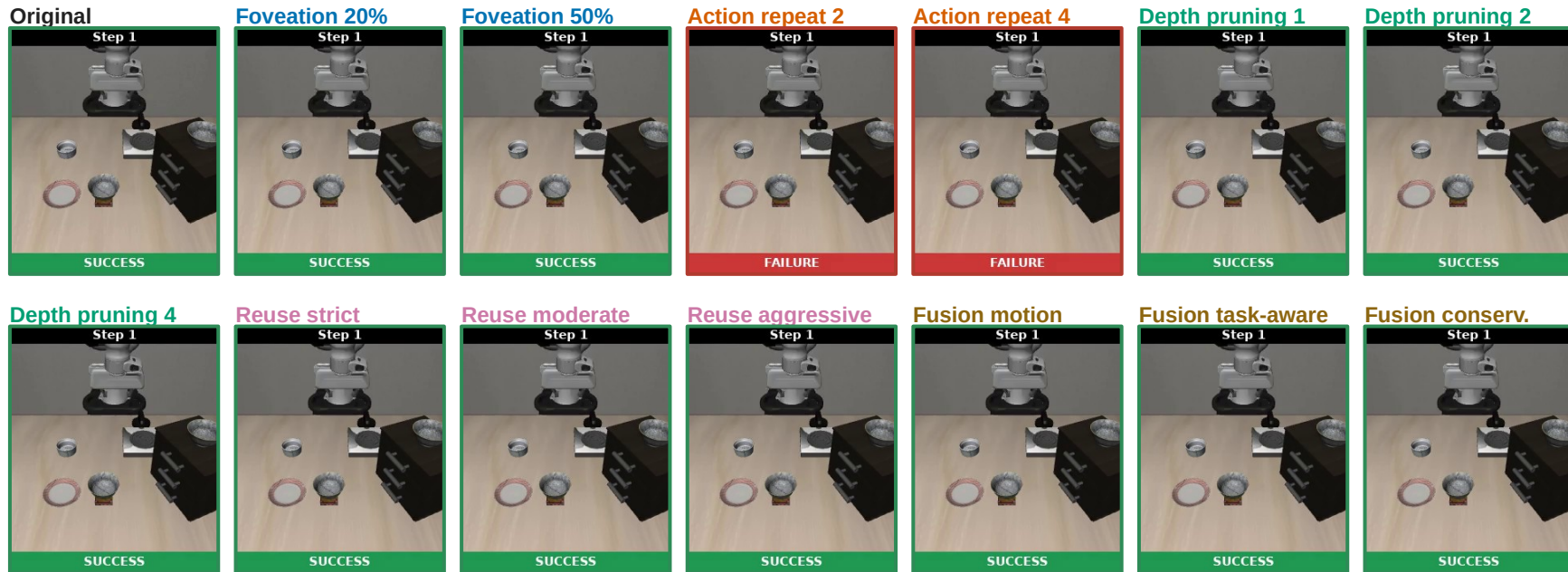
OpenVLA, Fractal: Open drawer

All 14 configurations on the same episode (same initial state), 2× speed. Green border = success, red = failure.



UniVLA, LIBERO Spatial: Bowl on cookie box to plate

All 14 configurations on the same episode, 3× speed, runs cut at the slide end. Green border = success, red = failure.



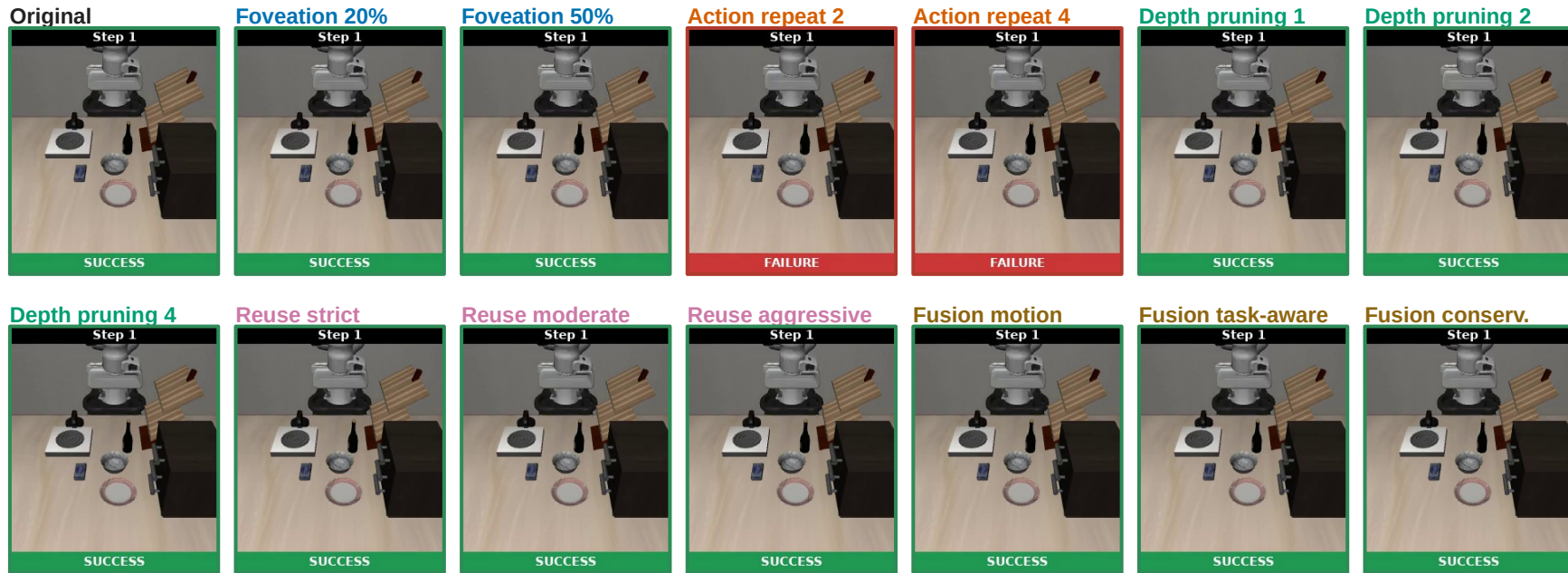
UniVLA, LIBERO Object: Orange juice into basket

All 14 configurations on the same episode, 4× speed, runs cut at the slide end. Green border = success, red = failure.



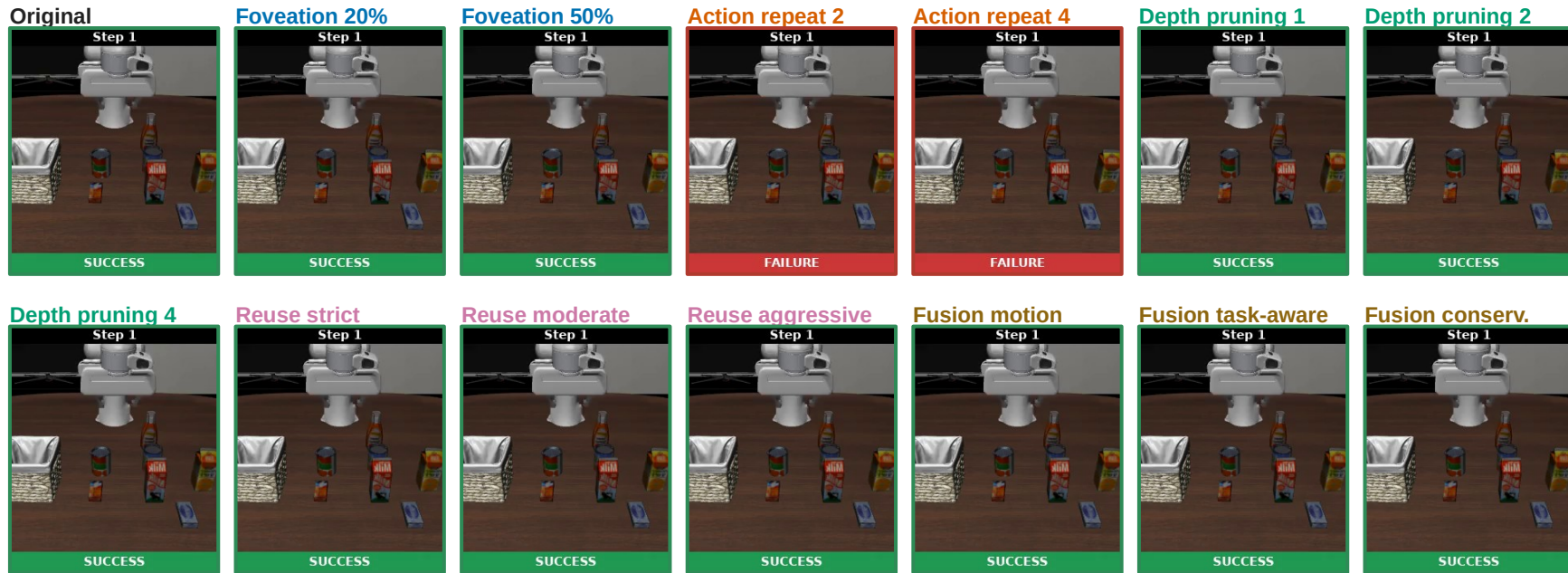
UniVLA, LIBERO Goal: Bowl on plate

All 14 configurations on the same episode, 3× speed, runs cut at the slide end. Green border = success, red = failure.

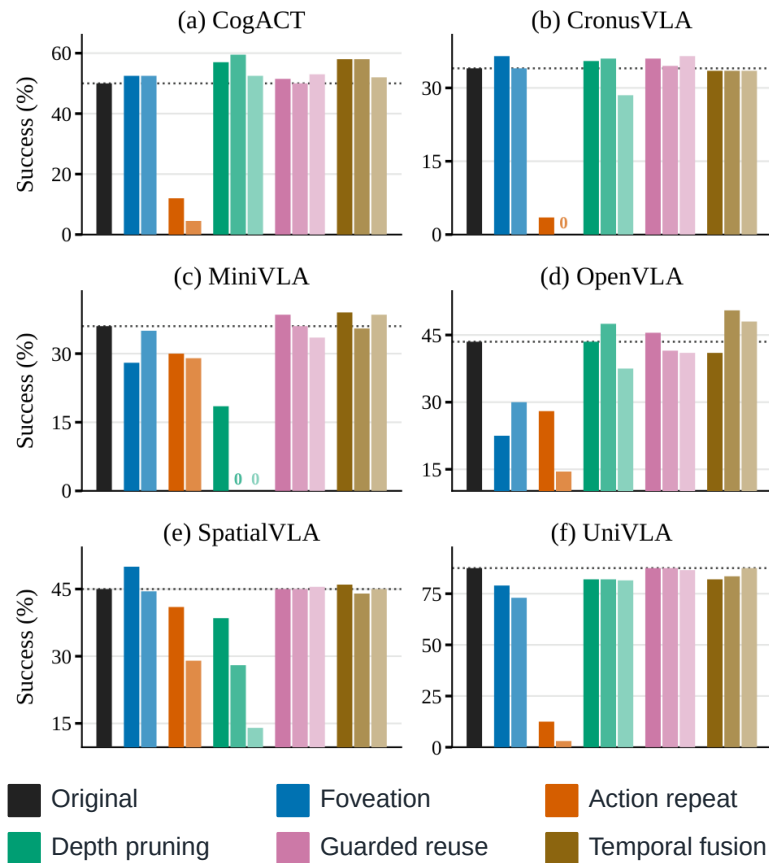


UniVLA, LIBERO Long: Cream cheese and butter into basket

All 14 configurations on the same episode, 8× speed, runs cut at the slide end. Green border = success, red = failure.



Which tricks preserve success?



Q1 Success

Guarded reuse and temporal fusion are the safest for preserving success.

Selected backbones also gain:

Foveation, WidowX: CogACT +2.5 · SpatialVLA +5.0 · CronusVLA +2.5

Depth pruning, WidowX: CogACT +9.5

Backbone-dependent:

Foveation, LIBERO Goal: UniVLA +7.0 vs SmoIVLA -13.0

Action repeat: mainly a speed baseline

Fig. 1: success per configuration, six WidowX backbones (14 bars each, darker to lighter within a trick). Dotted line = original policy, 0 = no successful episodes. Deltas from Tables I and II, best setting per trick.

Which tricks are actually faster?

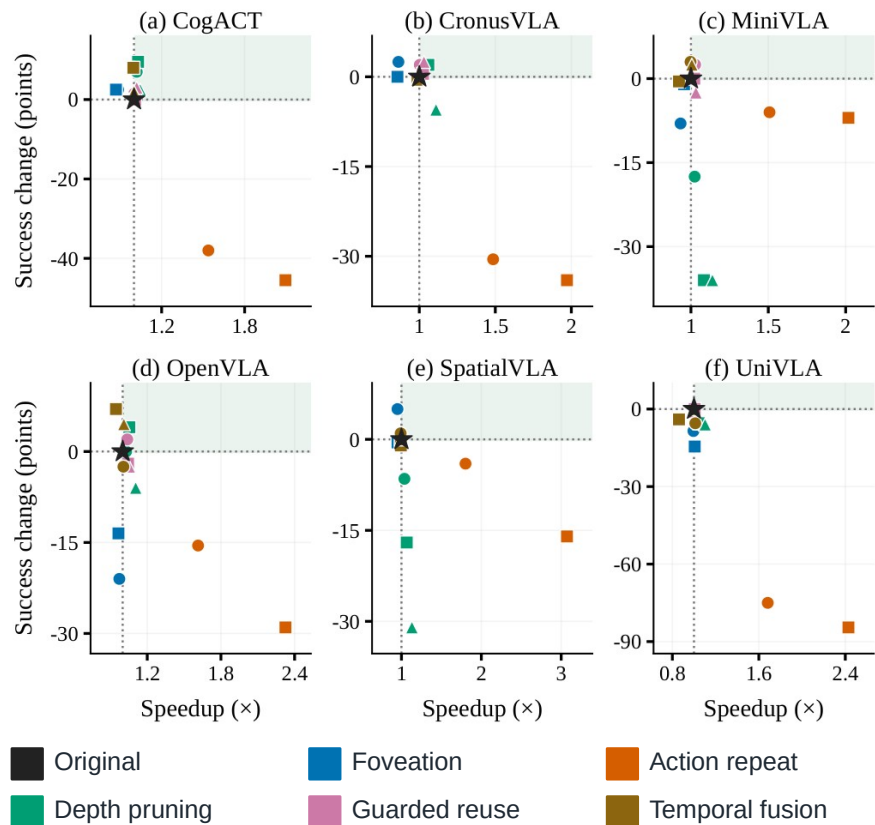


Fig. 4, SimplerEnv WidowX. Star = original, shaded = faster and no worse. Colour = trick; circle, square, triangle = first, second, third setting of that trick.

Q2 Speed

Action repeat gives the largest speedup, depth pruning gives decoder-level savings, guarded reuse gives the safest trade-off.

Latency, ms per step	Original → trick
Repeat · SpatialVLA WidowX	423.5 → 235.1
Repeat · OpenVLA LIBERO Long	171.6 → 93.2
Pruning · OpenVLA WidowX	210.2 → 199.4
Reuse · SmolVLA LIBERO Long	289.5 → 268.9

Takeaways

Guarded reuse

Most reliable. Preserves success, modest speedup when the gates open.

Action repeat

Largest speedup, but the risk of long open-loop execution.

Depth pruning

Decoder-level savings, benefit depends on the backbone.

Foveation · Temporal fusion

Test how much the visual representation can change while the policy stays stable.

VLA efficiency is policy- and environment-dependent. Evaluate every trick under the target backbone and environment before applying it.